

Entrega 2. Operaciones SQL principales mediante SQLJ

Esta entrega aplica los fundamentos anteriores a las operaciones SQL más habituales. Para cada operación se separan claramente la sentencia SQL convencional y su integración dentro de Java mediante SQLJ.

Los ejemplos utilizan un contexto SQLJ explícito llamado contexto. Cuando no se especifica un contexto, SQLJ utiliza normalmente el contexto predeterminado. Tanto Oracle como IBM documentan las cláusulas ejecutables SQLJ con la forma `#sql [contexto] { sentencia };`.

[\[ibm.com\]](#), [\[ibm.com\]](#)

1. Preparación común

1.1. Modelo de datos

```
1  CLIENTE
2  |— id_cliente
3  |— nombre
4  |— email
5  |— ciudad
6
7  PEDIDO
8  |— id_pedido
9  |— id_cliente
10 |— fecha
11 |— importe
12 |— estado
13
14 PRODUCTO
15 |— id_producto
16 |— nombre
17 |— precio
18 |— categoria
19
20 LINEA_PEDIDO
21 |— id_pedido
22 |— id_producto
23 |— cantidad
```

Relaciones:

```

1 CLIENTE 1 ——— N PEDIDO
2
3 PEDIDO 1 ——— N LINEA_PEDIDO
4
5 PRODUCTO 1 ——— N LINEA_PEDIDO

```

1.2. Datos de ejemplo

CLIENTE

1				
2	id_cliente	nombre	email	ciudad
3				
4	1	Ana	ana@example.com	Bilbao
5	2	Luis	luis@example.com	Madrid
6	3	Marta	marta@example.com	Bilbao
7	4	Pablo	pablo@example.com	Sevilla
8				

PEDIDO

1					
2	id_pedido	id_cliente	fecha	importe	estado
3					
4	101	1	2026-08-01	120.00	PENDIENTE
5	102	1	2026-08-03	85.50	ENVIADO
6	103	3	2026-08-04	210.00	ENTREGADO
7	104	2	2026-08-05	NULL	PENDIENTE
8					

PRODUCTO

1				
2	id_producto	nombre	precio	categoria
3				
4	10	Teclado	40.00	Periféricos
5	11	Ratón	25.00	Periféricos
6	12	Monitor	180.00	Pantallas
7	13	Adaptador	15.00	Accesorios
8				

LINEA_PEDIDO

1			
---	--	--	--

2	id_pedido	id_producto	cantidad
3			
4	101	10	2
5	101	11	1
6	102	12	1
7	103	10	1
8	103	12	1
9			

1.3. Tipos e iteradores utilizados

Los iteradores SQLJ se emplean para recibir resultados de varias filas y son conceptualmente similares a un ResultSet de JDBC. Oracle e IBM distinguen entre iteradores nombrados y posicionales. [\[docs.cloud...oracle.com\]](https://docs.oracle.com/), [\[ibm.com\]](https://ibm.com/)

```

1  import java.math.BigDecimal;
2  import java.sql.Date;
3
4  #sql iterator ClienteIterator(
5      int idCliente,
6      String nombre,
7      String email,
8      String ciudad
9  );
10
11 #sql iterator PedidoIterator(
12     int idPedido,
13     Date fecha,
14     BigDecimal importe,
15     String estado
16 );
17
18 #sql iterator ClientePedidoIterator(
19     int idCliente,
20     String nombreCliente,
21     Integer idPedido,
22     Date fecha,
23     BigDecimal importe,
24     String estado
25 );
26

```

```

27 #sql iterator ProductoCantidadIterator(
28     int idPedido,
29     String cliente,
30     String producto,
31     int cantidad,
32     BigDecimal precioUnitario,
33     BigDecimal subtotal
34 );
35
36 #sql iterator CiudadIterator(
37     String ciudad
38 );
39
40 #sql iterator ResumenClienteIterator(
41     int idCliente,
42     String nombreCliente,
43     long numeroPedidos,
44     BigDecimal importeTotal,
45     BigDecimal importeMedio
46 );

```

Se utilizan clases envoltorio, como Integer, para columnas que pueden ser NULL, especialmente en resultados procedentes de un LEFT JOIN.

2. SELECT de una fila

2.1. ¿Qué hace?

Recupera una única fila de la base de datos y asigna sus columnas a variables Java.

Debe utilizarse cuando la consulta devuelve exactamente una fila.

2.2. Representación gráfica

```

1  idCliente = 1
2      |
3      ▼
4  CLIENTE
5
6  ┌──────────┬────────┬────────┬────────┐
6  │ id_cliente │ nombre │ ciudad │         │

```

7			
8	1	Ana	Bilbao
9			
10			
11	▼		
12	nombreCliente = "Ana"		
13	ciudadCliente = "Bilbao"		

2.3. SQL aislado

```

1  SELECT nombre, ciudad
2  FROM cliente
3  WHERE id_cliente = 1;

```

2.4. Implementación SQLJ

```

1  int idCliente = 1;
2  String nombreCliente = null;
3  String ciudadCliente = null;
4
5  #sql [contexto] {
6      SELECT nombre, ciudad
7      INTO :nombreCliente, :ciudadCliente
8      FROM cliente
9      WHERE id_cliente = :idCliente
10 };

```

El INTO identifica las variables de salida. La variable idCliente actúa como variable host de entrada.

2.5. Resultado esperado

```

1  nombreCliente = "Ana"
2  ciudadCliente = "Bilbao"

```

2.6. Errores habituales

- La consulta no devuelve filas.
- La consulta devuelve más de una fila.
- El orden de las variables de salida no coincide con el de las columnas.

- Una columna devuelve NULL y la variable Java no puede representarlo.
 - Los tipos SQL y Java no son compatibles.
 - Se emplea SELECT INTO con un filtro que no garantiza unicidad.
-

2.7. Recomendación práctica

Utilizar SELECT INTO con:

- Claves primarias.
- Claves únicas.
- Funciones de agregación que garanticen una fila.
- Consultas cuya cardinalidad esté claramente controlada.

No debe utilizarse para una consulta que pueda devolver varios clientes.

3. SELECT de varias filas

3.1. ¿Qué hace?

Recupera cero, una o varias filas. En SQLJ, los resultados se procesan mediante un iterador.

3.2. Representación gráfica

```
1  SELECT clientes de Bilbao
2      |
3      ▼
4      ┌───┬───┐
5      │ 1 │ Ana │
6      │ 3 │ Marta │
7      └───┴───┘
8      |
9      ▼
10  ClienteIterator
11  |
12  └─ next() → Ana
13  └─ next() → Marta
14  └─ next() → false
```

3.3. SQL aislado

```
1 SELECT id_cliente, nombre, email, ciudad
2 FROM cliente
3 WHERE ciudad = 'Bilbao'
4 ORDER BY nombre;
```

3.4. Implementación SQLJ

```
1 String ciudadBuscada = "Bilbao";
2 ClienteIterator clientes = null;
3
4 try {
5     #sql [contexto] clientes = {
6         SELECT id_cliente AS idCliente,
7             nombre AS nombre,
8             email AS email,
9             ciudad AS ciudad
10        FROM cliente
11        WHERE ciudad = :ciudadBuscada
12        ORDER BY nombre
13    };
14
15    while (clientes.next()) {
16        System.out.println(
17            clientes.idCliente() + " | " +
18            clientes.nombre() + " | " +
19            clientes.email() + " | " +
20            clientes.ciudad()
21        );
22    }
23 } finally {
24     if (clientes != null) {
25         clientes.close();
26     }
27 }
```

3.5. Resultado esperado

1	1	Ana	ana@example.com	Bilbao
---	---	-----	-----------------	--------

3.6. Errores habituales

- No cerrar el iterador.
 - Usar nombres de columnas que no coinciden con el iterador.
 - No considerar columnas que pueden contener NULL.
 - Suponer que el orden de las filas está garantizado sin ORDER BY.
 - Recuperar un volumen excesivo de datos.
-

3.7. Recomendación práctica

Usar:

- Alias explícitos.
- Filtros suficientemente selectivos.
- ORDER BY si el orden tiene significado funcional.
- Paginación cuando el resultado pueda ser grande.

La sintaxis de paginación cambia entre bases de datos y debe verificarse para cada fabricante.

4. INSERT

4.1. ¿Qué hace?

Añade una nueva fila a una tabla.

4.2. Representación gráfica

```
1 Variables Java
2 _____
3 idCliente = 5
4 nombre = "Elena"
5 email = "elena@example.com"
6 ciudad = "Vitoria"
7         |
8         ▼
```



```
9          INSERT
10         |
11         ▼
12 Nueva fila en CLIENTE
```

4.3. SQL aislado

```
1  INSERT INTO cliente (
2      id_cliente,
3      nombre,
4      email,
5      ciudad
6  )
7  VALUES (
8      5,
9      'Elena',
10     'elena@example.com',
11     'Vitoria'
12 );
```

4.4. Implementación SQLJ

```
1  int idCliente = 5;
2  String nombre = "Elena";
3  String email = "elena@example.com";
4  String ciudad = "Vitoria";
5
6  #sql [contexto] {
7      INSERT INTO cliente (
8          id_cliente,
9          nombre,
10         email,
11         ciudad
12     )
13     VALUES (
14         :idCliente,
15         :nombre,
16         :email,
17         :ciudad
18     )
```

```
19 };
```

4.5. Resultado esperado

```
1 CLIENTE
```

```
2
```

```
3
```

4	id_cliente	nombre	email	ciudad
5				
6	5	Elena	elena@example.com	Vitoria
7				

La fila queda pendiente de confirmación si el autocommit está desactivado.

4.6. Errores habituales

- Clave primaria duplicada.
 - Columna obligatoria sin valor.
 - Longitud superior al tamaño permitido.
 - Tipo incompatible.
 - Formato de fecha incorrecto.
 - Violación de una restricción CHECK.
 - No confirmar la transacción.
 - Confirmar demasiado pronto y romper una operación de negocio.
-

4.7. Recomendación práctica

Indicar siempre las columnas:

```
1 INSERT INTO cliente (  
2     id_cliente,  
3     nombre,  
4     email,  
5     ciudad  
6 )  
7 VALUES (...);
```

Evitar:

```
1 INSERT INTO cliente  
2 VALUES (...);
```

La primera forma resiste mejor cambios en el orden físico o lógico de las columnas.

5. UPDATE

5.1. ¿Qué hace?

Modifica una o varias filas existentes.

5.2. Representación gráfica

```
1  PEDIDO 101
2
3  Estado anterior
4  PENDIENTE
5      |
6      | UPDATE
7      ▼
8  Estado nuevo
9  ENVIADO
```

5.3. SQL aislado

```
1  UPDATE pedido
2  SET estado = 'ENVIADO'
3  WHERE id_pedido = 101;
```

5.4. Implementación SQLJ

```
1  int idPedido = 101;
2  String nuevoEstado = "ENVIADO";
3
4  #sql [contexto] {
5      UPDATE pedido
6      SET estado = :nuevoEstado
7      WHERE id_pedido = :idPedido
8  };
```

5.5. Resultado esperado

1		
2	id_pedido	estado
3		
4	101	ENVIADO
5		

5.6. Errores habituales

- Omitir la cláusula WHERE.
 - Utilizar un filtro demasiado amplio.
 - Actualizar cero filas y tratarlo como éxito.
 - Sobrescribir cambios realizados por otro proceso.
 - Asignar un estado no permitido.
 - Confirmar la transacción antes de terminar la operación.
 - No comprobar el número de filas afectadas cuando sea relevante.
-

5.7. Recomendación práctica

Antes de ejecutar actualizaciones sensibles:

1. Comprobar el filtro mediante un SELECT.
2. Utilizar una clave primaria o una condición claramente controlada.
3. Incluir el estado previo si se necesita control de concurrencia.

Ejemplo:

```
1 String estadoAnterior = "PENDIENTE";
2 String nuevoEstado = "ENVIADO";
3
4 #sql [contexto] {
5     UPDATE pedido
6     SET estado = :nuevoEstado
7     WHERE id_pedido = :idPedido
8     AND estado = :estadoAnterior
9 };
```

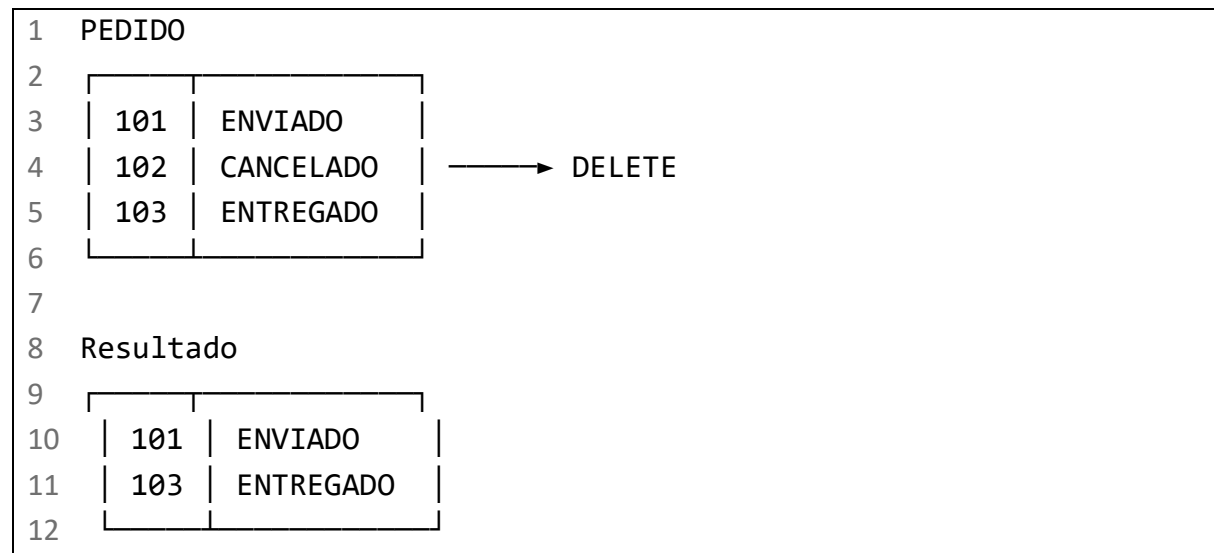
Este patrón evita actualizar un pedido cuyo estado ya ha cambiado desde que fue leído.

6. DELETE

6.1. ¿Qué hace?

Elimina una o varias filas.

6.2. Representación gráfica



6.3. SQL aislado

```
1 DELETE FROM pedido
2 WHERE id_pedido = 102;
```

6.4. Implementación SQLJ

```
1 int idPedido = 102;
2
3 #sql [contexto] {
4     DELETE FROM pedido
5     WHERE id_pedido = :idPedido
6 };
```

6.5. Resultado esperado

El pedido desaparece de la tabla si:

- Existe.

- No está referenciado por filas que impidan la eliminación.
 - La transacción se confirma.
-

6.6. Errores habituales

- Omitir WHERE.
 - Eliminar un pedido con líneas asociadas.
 - Violación de clave ajena.
 - Eliminar cero filas sin detectarlo.
 - Confiar en borrado en cascada sin verificar el esquema.
 - Confirmar el borrado antes de completar otras operaciones relacionadas.
-

6.7. Recomendación práctica

Eliminar primero las entidades dependientes cuando no exista una regla de borrado en cascada:

```
1  #sql [contexto] {
2      DELETE FROM linea_pedido
3      WHERE id_pedido = :idPedido
4  };
5
6  #sql [contexto] {
7      DELETE FROM pedido
8      WHERE id_pedido = :idPedido
9  };
```

Ambas sentencias deben ejecutarse dentro de la misma transacción.

7. INNER JOIN

7.1. ¿Qué hace?

Devuelve únicamente las filas que tienen correspondencia en ambas tablas.

7.2. Representación gráfica

1	CLIENTE	PEDIDO
---	---------	--------

2					
3	id_cliente	nombre		id_pedido	id_cliente
4					
5	1	Ana	↔	101	1
6	2	Luis	↔	104	2
7	3	Marta	↔	103	3
8	4	Pablo			
9					

Resultado:

1		
2	cliente	id_pedido
3		
4	Ana	101
5	Ana	102
6	Luis	104
7	Marta	103
8		

Pablo no aparece porque no tiene pedidos.

7.3. SQL aislado

```

1  SELECT c.id_cliente,
2         c.nombre,
3         p.id_pedido,
4         p.fecha,
5         p.importe,
6         p.estado
7  FROM cliente c
8  INNER JOIN pedido p
9         ON p.id_cliente = c.id_cliente
10 ORDER BY c.id_cliente, p.id_pedido;

```

7.4. Implementación SQLJ

```

1  ClientePedidoIterator resultados = null;
2
3  try {
4      #sql [contexto] resultados = {
5          SELECT c.id_cliente AS idCliente,

```

```

6         c.nombre AS nombreCliente,
7         p.id_pedido AS idPedido,
8         p.fecha AS fecha,
9         p.importe AS importe,
10        p.estado AS estado
11    FROM cliente c
12    INNER JOIN pedido p
13        ON p.id_cliente = c.id_cliente
14    ORDER BY c.id_cliente, p.id_pedido
15 };
16
17 while (resultados.next()) {
18     System.out.println(
19         resultados.nombreCliente() + " | " +
20         resultados.idPedido() + " | " +
21         resultados.estado()
22     );
23 }
24 } finally {
25     if (resultados != null) {
26         resultados.close();
27     }
28 }

```

7.5. Resultado esperado

1	Ana		101		PENDIENTE
2	Ana		102		ENVIADO
3	Luis		104		PENDIENTE
4	Marta		103		ENTREGADO

7.6. Errores habituales

- Omitir la condición ON.
- Relacionar columnas incorrectas.
- Generar un producto cartesiano.
- No cualificar columnas con el mismo nombre.
- Confundir INNER JOIN con LEFT JOIN.
- Filtrar en el lugar incorrecto.
- Recuperar columnas innecesarias.

7.7. Recomendación práctica

Usar alias cortos y significativos:

```
1 FROM cliente c
2 INNER JOIN pedido p
3     ON p.id_cliente = c.id_cliente
```

Las columnas usadas en filtros y relaciones deberían disponer de índices apropiados cuando el volumen de datos lo justifique.

8. LEFT JOIN

8.1. ¿Qué hace?

Devuelve todas las filas de la tabla izquierda y las coincidencias de la tabla derecha.

Cuando no existe coincidencia, las columnas de la tabla derecha contienen NULL.

8.2. Representación gráfica

1	CLIENTE			PEDIDO	
2					
3	id_cliente	nombre		id_pedido	id_cliente
4					
5	1	Ana	↔	101	1
6	2	Luis	↔	104	2
7	3	Marta	↔	103	3
8	4	Pablo			
9					

Resultado:

1		
2	cliente	id_pedido
3		
4	Ana	101
5	Ana	102
6	Luis	104
7	Marta	103
8	Pablo	NULL

8.3. SQL aislado

```
1  SELECT c.id_cliente,  
2         c.nombre,  
3         p.id_pedido,  
4         p.fecha,  
5         p.importe,  
6         p.estado  
7  FROM cliente c  
8  LEFT JOIN pedido p  
9      ON p.id_cliente = c.id_cliente  
10 ORDER BY c.id_cliente, p.id_pedido;
```

8.4. Implementación SQLJ

```
1  ClientePedidoIterator resultados = null;  
2  
3  try {  
4      #sql [contexto] resultados = {  
5          SELECT c.id_cliente AS idCliente,  
6                 c.nombre AS nombreCliente,  
7                 p.id_pedido AS idPedido,  
8                 p.fecha AS fecha,  
9                 p.importe AS importe,  
10                p.estado AS estado  
11          FROM cliente c  
12          LEFT JOIN pedido p  
13              ON p.id_cliente = c.id_cliente  
14          ORDER BY c.id_cliente, p.id_pedido  
15      };  
16  
17      while (resultados.next()) {  
18          Integer idPedido = resultados.idPedido();  
19  
20          System.out.println(  
21              resultados.nombreCliente() + " | " +  
22              (idPedido == null ? "Sin pedidos" : idPedido)  
23          );  
24      }  
25  } catch (SQLException e) {  
26      e.printStackTrace();  
27  }
```

```
24     }
25 } finally {
26     if (resultados != null) {
27         resultados.close();
28     }
29 }
```

8.5. Resultado esperado

Pablo aparece aunque no tenga pedidos:

1 Pablo Sin pedidos

8.6. Errores habituales

- Declarar idPedido como int en vez de Integer.
- Colocar en WHERE un filtro de la tabla derecha.
- Convertir accidentalmente el LEFT JOIN en un comportamiento equivalente a INNER JOIN.
- No distinguir entre ausencia de pedido y un valor nulo almacenado.
- Aplicar funciones sin tratar NULL.

Ejemplo problemático:

```
1 FROM cliente c
2 LEFT JOIN pedido p
3     ON p.id_cliente = c.id_cliente
4 WHERE p.estado = 'PENDIENTE'
```

Los clientes sin pedidos desaparecen porque p.estado es NULL.

Si se quieren conservar todos los clientes, el filtro debe formar parte del ON:

```
1 FROM cliente c
2 LEFT JOIN pedido p
3     ON p.id_cliente = c.id_cliente
4     AND p.estado = 'PENDIENTE'
```

8.7. Recomendación práctica

En un LEFT JOIN:

- Usar clases envoltorio para columnas anulables.
- Decidir conscientemente si un filtro pertenece a ON o a WHERE.

- Documentar qué significa una fila sin correspondencia.
-

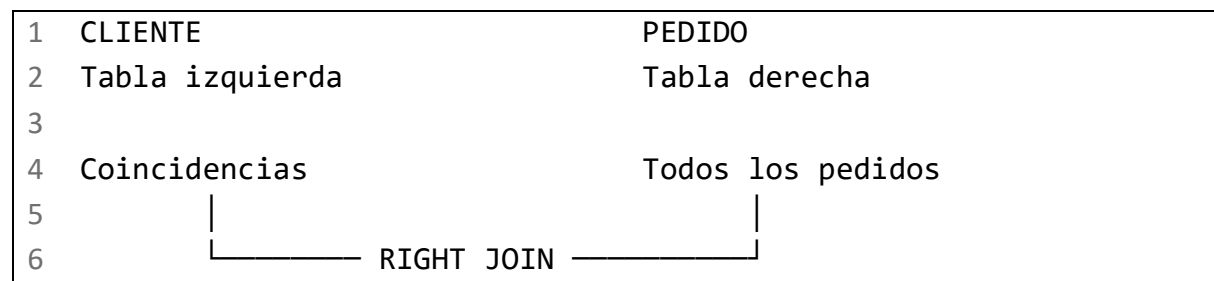
9. RIGHT JOIN

9.1. ¿Qué hace?

Devuelve todas las filas de la tabla derecha y las coincidencias de la tabla izquierda.

Es la operación simétrica de LEFT JOIN.

9.2. Representación gráfica



9.3. SQL aislado

```
1 SELECT c.nombre,  
2       p.id_pedido,  
3       p.estado  
4 FROM cliente c  
5 RIGHT JOIN pedido p  
6     ON p.id_cliente = c.id_cliente;
```

9.4. Implementación SQLJ

```
1 ClientePedidoIterator resultados = null;  
2  
3 try {  
4     #sql [contexto] resultados = {  
5         SELECT c.id_cliente AS idCliente,  
6               c.nombre AS nombreCliente,  
7               p.id_pedido AS idPedido,  
8               p.fecha AS fecha,  
9               p.importe AS importe,
```

```

10         p.estado AS estado
11     FROM cliente c
12     RIGHT JOIN pedido p
13         ON p.id_cliente = c.id_cliente
14     ORDER BY p.id_pedido
15 };
16
17     while (resultados.next()) {
18         System.out.println(
19             resultados.idPedido() + " | " +
20             resultados.nombreCliente()
21         );
22     }
23 } finally {
24     if (resultados != null) {
25         resultados.close();
26     }
27 }

```

9.5. Resultado esperado

Todos los pedidos aparecen, incluso si la relación permitiera pedidos sin cliente.

En el modelo propuesto, una clave ajena correctamente definida normalmente impedirá que exista un pedido con un id_cliente inexistente.

9.6. Errores habituales

- Confundir qué tabla conserva todas las filas.
 - Usar RIGHT JOIN cuando un LEFT JOIN sería más fácil de leer.
 - Suponer que todas las bases de datos admiten exactamente las mismas variantes de JOIN.
 - No considerar que las columnas de la tabla izquierda pueden quedar a NULL.
-

9.7. Recomendación práctica

Cuando exista alguna duda de portabilidad o legibilidad, intercambiar las tablas y usar LEFT JOIN.

Equivalente conceptual:

```

1  SELECT c.nombre,
2         p.id_pedido,
3         p.estado
4  FROM pedido p
5  LEFT JOIN cliente c
6        ON c.id_cliente = p.id_cliente;

```

El soporte concreto de RIGHT JOIN debe comprobarse en la versión y el dialecto de la base de datos utilizada.

10. JOIN entre tres o más tablas

10.1. ¿Qué hace?

Combina información relacionada de varias tablas.

Ejemplo:

```

1  CLIENTE
2    |
3    ▼
4  PEDIDO
5    |
6    ▼
7  LINEA_PEDIDO
8    |
9    ▼
10 PRODUCTO

```

10.2. Representación gráfica

```

1  Ana
2  |
3  └─ Pedido 101
4      |
5      └─ 2 × Teclado
6          2 × 40.00 = 80.00
7
8      └─ 1 × Ratón
9          1 × 25.00 = 25.00

```

10.3. SQL aislado

```
1  SELECT p.id_pedido,  
2         c.nombre AS cliente,  
3         pr.nombre AS producto,  
4         lp.cantidad,  
5         pr.precio AS precio_unitario,  
6         lp.cantidad * pr.precio AS subtotal  
7  FROM cliente c  
8  INNER JOIN pedido p  
9      ON p.id_cliente = c.id_cliente  
10 INNER JOIN linea_pedido lp  
11     ON lp.id_pedido = p.id_pedido  
12 INNER JOIN producto pr  
13     ON pr.id_producto = lp.id_producto  
14 ORDER BY p.id_pedido, pr.nombre;
```

10.4. Implementación SQLJ

```
1  ProductoCantidadIterator lineas = null;  
2  
3  try {  
4      #sql [contexto] lineas = {  
5          SELECT p.id_pedido AS idPedido,  
6                 c.nombre AS cliente,  
7                 pr.nombre AS producto,  
8                 lp.cantidad AS cantidad,  
9                 pr.precio AS precioUnitario,  
10                lp.cantidad * pr.precio AS subtotal  
11          FROM cliente c  
12          INNER JOIN pedido p  
13              ON p.id_cliente = c.id_cliente  
14          INNER JOIN linea_pedido lp  
15              ON lp.id_pedido = p.id_pedido  
16          INNER JOIN producto pr  
17              ON pr.id_producto = lp.id_producto  
18          ORDER BY p.id_pedido, pr.nombre  
19      };  
20  
21      while (lineas.next()) {  
22          System.out.println(  

```

```

23         lineas.idPedido() + " | " +
24         lineas.cliente() + " | " +
25         lineas.producto() + " | " +
26         lineas.cantidad() + " | " +
27         lineas.subtotal()
28     );
29 }
30 } finally {
31     if (lineas != null) {
32         lineas.close();
33     }
34 }

```

10.5. Resultado esperado

1	101	Ana	Ratón	1	25.00
2	101	Ana	Teclado	2	80.00
3	102	Ana	Monitor	1	180.00
4	103	Marta	Monitor	1	180.00
5	103	Marta	Teclado	1	40.00

10.6. Errores habituales

- Omitir una relación.
- Relacionar id_producto con id_pedido.
- Generar duplicados no comprendidos.
- Calcular importes con tipos de precisión insuficiente.
- No utilizar alias.
- Aplicar filtros después de producir un resultado demasiado grande.

10.7. Recomendación práctica

Construir el JOIN de forma incremental:

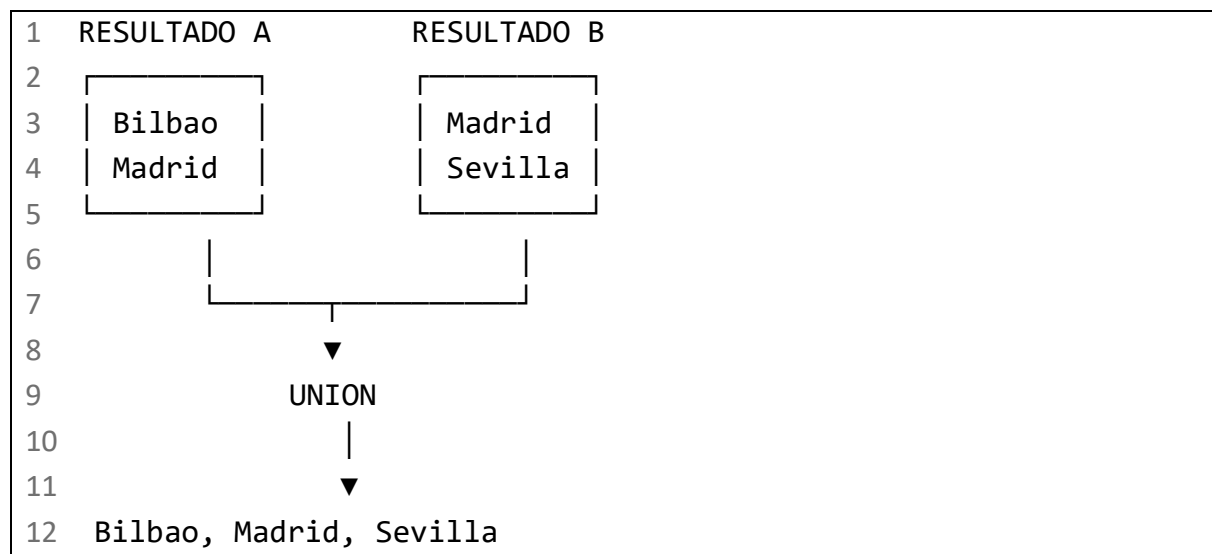
1. Probar CLIENTE con PEDIDO.
2. Añadir LINEA_PEDIDO.
3. Añadir PRODUCTO.
4. Verificar el número de filas en cada paso.
5. Añadir después cálculos y filtros.

11. UNION

11.1. ¿Qué hace?

Combina los resultados de dos consultas y elimina las filas duplicadas.

11.2. Representación gráfica



11.3. SQL aislado

```
1 SELECT ciudad
2 FROM cliente
3 WHERE id_cliente <= 2
4
5 UNION
6
7 SELECT ciudad
8 FROM cliente
9 WHERE id_cliente >= 2;
```

11.4. Implementación SQLJ

```
1 CiudadIterator ciudades = null;
2
3 try {
```

```
4      #sql [contexto] ciudades = {
5          SELECT ciudad AS ciudad
6          FROM cliente
7          WHERE id_cliente <= 2
8
9          UNION
10
11         SELECT ciudad AS ciudad
12         FROM cliente
13         WHERE id_cliente >= 2
14
15         ORDER BY ciudad
16     };
17
18     while (ciudades.next()) {
19         System.out.println(ciudades.ciudad());
20     }
21 } finally {
22     if (ciudades != null) {
23         ciudades.close();
24     }
25 }
```

11.5. Resultado esperado

```
1  Bilbao
2  Madrid
3  Sevilla
```

Bilbao aparece una sola vez, aunque exista en más de una fila de origen.

11.6. Errores habituales

- Distinto número de columnas.
 - Tipos incompatibles.
 - Pensar que conserva duplicados.
 - Aplicar ORDER BY separadamente en cada consulta.
 - Utilizar UNION cuando no es necesario eliminar duplicados.
 - Confundir filas duplicadas con valores repetidos en una sola columna.
-

11.7. Recomendación práctica

Usar UNION únicamente cuando se necesite eliminar duplicados. Esta eliminación puede requerir trabajo adicional de ordenación o comparación.

Las consultas combinadas deben devolver:

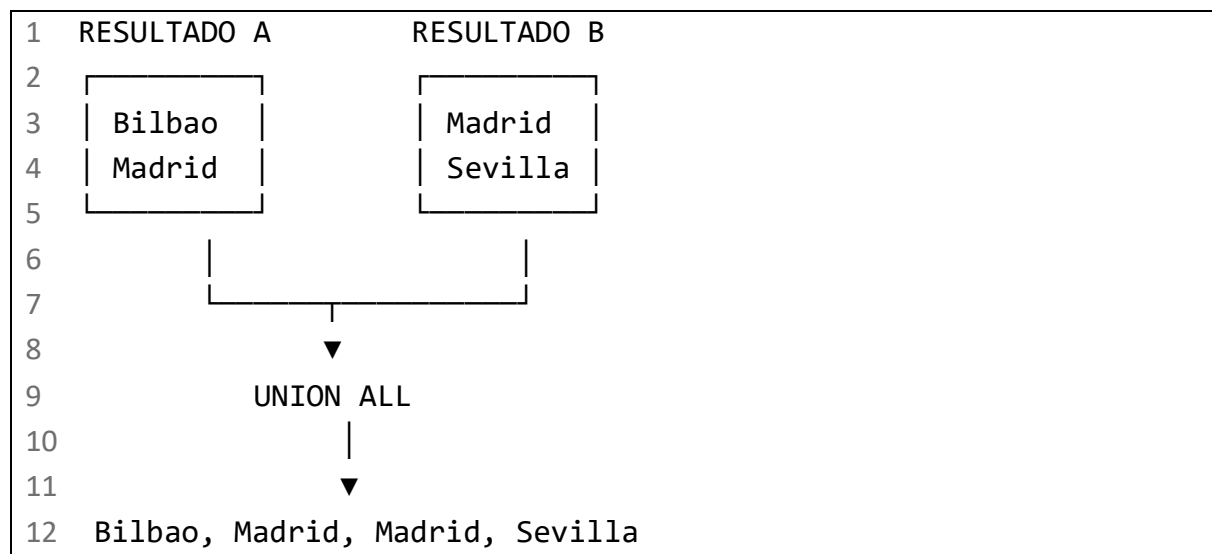
- El mismo número de columnas.
 - Tipos compatibles en posiciones equivalentes.
 - Significados lógicos equivalentes.
-

12. UNION ALL

12.1. ¿Qué hace?

Combina los resultados conservando las filas duplicadas.

12.2. Representación gráfica



12.3. SQL aislado

```
1 SELECT ciudad
2 FROM cliente
3 WHERE id_cliente <= 2
4
5 UNION ALL
6
```

```
7  SELECT ciudad
8  FROM cliente
9  WHERE id_cliente >= 2;
```

12.4. Implementación SQLJ

```
1  CiudadIterator ciudades = null;
2
3  try {
4      #sql [contexto] ciudades = {
5          SELECT ciudad AS ciudad
6          FROM cliente
7          WHERE id_cliente <= 2
8
9          UNION ALL
10
11         SELECT ciudad AS ciudad
12         FROM cliente
13         WHERE id_cliente >= 2
14     };
15
16     while (ciudades.next()) {
17         System.out.println(ciudades.ciudad());
18     }
19 } finally {
20     if (ciudades != null) {
21         ciudades.close();
22     }
23 }
```

12.5. Resultado esperado

```
1  Bilbao
2  Madrid
3  Madrid
4  Bilbao
5  Sevilla
```

El resultado exacto puede contener más repeticiones según los datos de cada consulta.

Sin un ORDER BY final no debe suponerse un orden determinado.

12.6. Errores habituales

- Usarlo cuando el negocio exige resultados únicos.
 - Confundir duplicados reales con filas procedentes de fuentes distintas.
 - Añadir DISTINCT después y anular su ventaja.
 - Suponer que ordena el resultado.
-

12.7. Recomendación práctica

Preferir UNION ALL cuando:

- Los conjuntos ya son disjuntos.
 - Se necesitan conservar repeticiones.
 - Se desea conocer cuántas veces aparece cada fila.
 - La eliminación de duplicados no aporta valor funcional.
-

13. Subconsultas

13.1. ¿Qué hacen?

Una subconsulta es una consulta contenida dentro de otra.

Ejemplo: localizar pedidos cuyo importe supera el importe medio.

13.2. Representación gráfica

```
1  PEDIDOS
2  120.00, 85.50, 210.00
3
4      ▼
5  AVG = 138.50
6      |
7      ▼
8  Pedidos con importe > 138.50
9      |
10     ▼
11  Pedido 103
```

Se ha omitido el importe NULL porque AVG normalmente ignora valores NULL.

13.3. SQL aislado

```
1  SELECT id_pedido,
2         fecha,
3         importe,
4         estado
5  FROM pedido
6  WHERE importe > (
7      SELECT AVG(importe)
8      FROM pedido
9  )
10 ORDER BY importe DESC;
```

13.4. Implementación SQLJ

```
1  PedidoIterator pedidos = null;
2
3  try {
4      #sql [contexto] pedidos = {
5          SELECT id_pedido AS idPedido,
6                 fecha AS fecha,
7                 importe AS importe,
8                 estado AS estado
9          FROM pedido
10         WHERE importe > (
11             SELECT AVG(importe)
12             FROM pedido
13         )
14         ORDER BY importe DESC
15     };
16
17     while (pedidos.next()) {
18         System.out.println(
19             pedidos.idPedido() + " | " +
20             pedidos.importe()
21         );
22     }
23 } finally {
24     if (pedidos != null) {
```

```
25         pedidos.close();
26     }
27 }
```

13.5. Resultado esperado

1	103		210.00
---	-----	--	--------

13.6. Errores habituales

- La subconsulta escalar devuelve más de una fila.
- Los tipos comparados no son compatibles.
- No considerar los valores NULL.
- Usar una subconsulta correlacionada costosa.
- Escribir una subconsulta cuando un JOIN sería más claro.
- Aplicar la agregación al conjunto equivocado.

13.7. Recomendación práctica

Antes de integrarla:

1. Ejecutar la subconsulta aisladamente.
2. Comprobar su cardinalidad.
3. Verificar el tratamiento de NULL.
4. Comparar su claridad con una solución mediante JOIN.

14. EXISTS

14.1. ¿Qué hace?

Comprueba si una subconsulta devuelve al menos una fila.

Es apropiado cuando interesa saber si existe una relación, no recuperar sus columnas.

14.2. Representación gráfica

1	CLIENTE Ana
2	

```
3      ▼
4  ¿Existe algún PEDIDO de Ana?
5      |
6      └─ Sí → incluir a Ana
7      └─ No → excluir a Ana
```

14.3. SQL aislado

```
1  SELECT c.id_cliente,
2         c.nombre,
3         c.email,
4         c.ciudad
5  FROM cliente c
6  WHERE EXISTS (
7      SELECT 1
8      FROM pedido p
9      WHERE p.id_cliente = c.id_cliente
10 );
```

14.4. Implementación SQLJ

```
1  ClienteIterator clientes = null;
2
3  try {
4      #sql [contexto] clientes = {
5          SELECT c.id_cliente AS idCliente,
6                 c.nombre AS nombre,
7                 c.email AS email,
8                 c.ciudad AS ciudad
9          FROM cliente c
10         WHERE EXISTS (
11             SELECT 1
12             FROM pedido p
13             WHERE p.id_cliente = c.id_cliente
14         )
15         ORDER BY c.id_cliente
16     };
17
18     while (clientes.next()) {
19         System.out.println(clientes.nombre());
20     }
21 }
```



```
20     }
21 } finally {
22     if (clientes != null) {
23         clientes.close();
24     }
25 }
```

14.5. Resultado esperado

```
1 Ana
2 Luis
3 Marta
```

Pablo no aparece porque no tiene pedidos.

14.6. Errores habituales

- Olvidar correlacionar la subconsulta.
 - Usar columnas innecesarias en lugar de SELECT 1.
 - Confundir EXISTS con un recuento.
 - Usar un JOIN que genera duplicados cuando solo se necesita comprobar existencia.
-

14.7. Recomendación práctica

Usar EXISTS cuando la pregunta funcional sea:

¿Existe al menos una fila relacionada que cumpla esta condición?

15. NOT EXISTS

15.1. ¿Qué hace?

Selecciona filas para las que no existe ninguna coincidencia en la subconsulta.

15.2. Representación gráfica

```
1 CLIENTE Pablo
2 |
```

```
3      ▼
4  ¿Existe algún pedido?
5      |
6      └─ No
7          |
8          ▼
9      Incluir a Pablo
```

15.3. SQL aislado

```
1  SELECT c.id_cliente,
2         c.nombre,
3         c.email,
4         c.ciudad
5  FROM cliente c
6  WHERE NOT EXISTS (
7      SELECT 1
8      FROM pedido p
9      WHERE p.id_cliente = c.id_cliente
10 );
```

15.4. Implementación SQLJ

```
1  ClienteIterator clientes = null;
2
3  try {
4      #sql [contexto] clientes = {
5          SELECT c.id_cliente AS idCliente,
6                 c.nombre AS nombre,
7                 c.email AS email,
8                 c.ciudad AS ciudad
9          FROM cliente c
10         WHERE NOT EXISTS (
11             SELECT 1
12             FROM pedido p
13             WHERE p.id_cliente = c.id_cliente
14         )
15     };
16
17     while (clientes.next()) {
```

```
18         System.out.println(clientes.nombre());
19     }
20 } finally {
21     if (clientes != null) {
22         clientes.close();
23     }
24 }
```

15.5. Resultado esperado

```
1 Pablo
```

15.6. Errores habituales

- Olvidar la condición de correlación.
- Usar NOT IN sin considerar valores NULL.
- Confundir ausencia de filas relacionadas con una columna relacionada a NULL.
- Añadir condiciones en el nivel equivocado.

15.7. Recomendación práctica

Para consultas anti-relación, como “clientes sin pedidos”, NOT EXISTS suele expresar con claridad la intención y evita ciertas complicaciones de NOT IN con valores NULL.

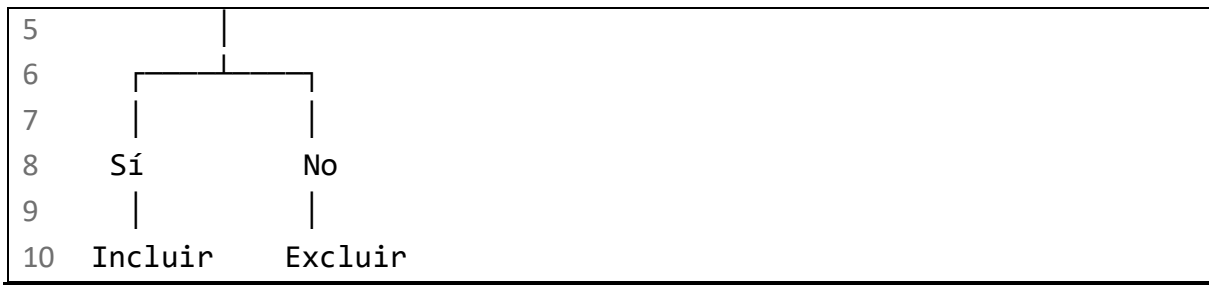
16. IN

16.1. ¿Qué hace?

Comprueba si un valor pertenece a una colección de valores o al resultado de una subconsulta.

16.2. Representación gráfica

```
1 ciudad del cliente
2     |
3     ▼
4 ¿Está en {Bilbao, Madrid}?
```



16.3. SQL aislado

```
1  SELECT id_cliente,  
2         nombre,  
3         email,  
4         ciudad  
5  FROM cliente  
6  WHERE ciudad IN ('Bilbao', 'Madrid');
```

16.4. Implementación SQLJ

Como la estructura SQLJ es estática, no se puede reemplazar una lista completa mediante una única variable host.

```
1  String ciudad1 = "Bilbao";  
2  String ciudad2 = "Madrid";  
3  
4  ClienteIterator clientes = null;  
5  
6  try {  
7      #sql [contexto] clientes = {  
8          SELECT id_cliente AS idCliente,  
9                 nombre AS nombre,  
10                email AS email,  
11                ciudad AS ciudad  
12            FROM cliente  
13            WHERE ciudad IN (:ciudad1, :ciudad2)  
14            ORDER BY nombre  
15        };  
16  
17        while (clientes.next()) {  
18            System.out.println(clientes.nombre());  
19        }  
20    } finally {
```

```
21     if (clientes != null) {  
22         clientes.close();  
23     }  
24 }
```

16.5. Resultado esperado

```
1 Ana  
2 Luis  
3 Marta
```

16.6. Errores habituales

- Intentar pasar una lista Java como una única variable host.
 - Incluir NULL en la lista sin comprender su efecto.
 - Construir una lista de longitud variable en SQL estático.
 - Utilizar una lista excesivamente grande.
 - Mezclar tipos incompatibles.
-

16.7. Recomendación práctica

Para una cantidad fija y pequeña de valores, usar variables host independientes.

Si el número de valores cambia durante la ejecución, considerar:

- JDBC con generación controlada de marcadores.
- Una tabla temporal.
- Una tabla de trabajo.
- Un tipo colección específico del fabricante.
- Una tabla de parámetros.

Las últimas alternativas dependen de la base de datos.

17. NOT IN

17.1. ¿Qué hace?

Comprueba que un valor no pertenezca a una colección.

17.2. Representación gráfica

```
1 Ciudad = Bilbao
2   |
3   ▼
4 ¿Está en {Madrid, Sevilla}?
5   |
6   └─ No
7       |
8       ▼
9       Incluir
```

17.3. SQL aislado

```
1 SELECT id_cliente,
2        nombre,
3        email,
4        ciudad
5 FROM cliente
6 WHERE ciudad NOT IN ('Madrid', 'Sevilla');
```

17.4. Implementación SQLJ

```
1 String ciudad1 = "Madrid";
2 String ciudad2 = "Sevilla";
3
4 ClienteIterator clientes = null;
5
6 try {
7     #sql [contexto] clientes = {
8         SELECT id_cliente AS idCliente,
9                nombre AS nombre,
10               email AS email,
11               ciudad AS ciudad
12     FROM cliente
13     WHERE ciudad NOT IN (:ciudad1, :ciudad2)
14 };
15
16 while (clientes.next()) {
17     System.out.println(clientes.nombre());
18 }
```

```
18     }
19 } finally {
20     if (clientes != null) {
21         clientes.close();
22     }
23 }
```

17.5. Resultado esperado

```
1 Ana
2 Marta
```

17.6. Errores habituales

El problema principal es NULL.

```
1 WHERE id_cliente NOT IN (
2     SELECT id_cliente
3     FROM pedido
4 )
```

Si la subconsulta devolviera algún NULL, la comparación podría resultar desconocida y no devolver las filas esperadas.

17.7. Recomendación práctica

Para anti-relaciones, preferir:

```
1 WHERE NOT EXISTS (
2     SELECT 1
3     FROM pedido p
4     WHERE p.id_cliente = c.id_cliente
5 )
```

Usar NOT IN cuando la lista sea pequeña, conocida y no contenga NULL.

18. GROUP BY

18.1. ¿Qué hace?

Agrupar filas que comparten uno o varios valores para calcular información resumida.

18.2. Representación gráfica

```
1 Pedidos
2 _____
3 Ana    120.00
4 Ana    85.50
5 Luis    NULL
6 Marta  210.00
7         |
8         ▼
9 GROUP BY cliente
10        |
11        ▼
12 Ana    → 2 pedidos → 205.50
13 Luis   → 1 pedido  → NULL
14 Marta → 1 pedido  → 210.00
```

18.3. SQL aislado

```
1 SELECT c.id_cliente,
2        c.nombre,
3        COUNT(p.id_pedido) AS numero_pedidos,
4        SUM(p.importe) AS importe_total,
5        AVG(p.importe) AS importe_medio
6 FROM cliente c
7 INNER JOIN pedido p
8     ON p.id_cliente = c.id_cliente
9 GROUP BY c.id_cliente, c.nombre
10 ORDER BY c.id_cliente;
```

18.4. Implementación SQLJ

```
1 ResumenClienteIterator resumen = null;
2
3 try {
4     #sql [contexto] resumen = {
5         SELECT c.id_cliente AS idCliente,
6                c.nombre AS nombreCliente,
7                COUNT(p.id_pedido) AS numeroPedidos,
```



```

8          SUM(p.importe) AS importeTotal,
9          AVG(p.importe) AS importeMedio
10         FROM cliente c
11         INNER JOIN pedido p
12             ON p.id_cliente = c.id_cliente
13         GROUP BY c.id_cliente, c.nombre
14         ORDER BY c.id_cliente
15     };
16
17     while (resumen.next()) {
18         System.out.println(
19             resumen.nombreCliente() + " | " +
20             resumen.numeroPedidos() + " | " +
21             resumen.importeTotal()
22         );
23     }
24 } finally {
25     if (resumen != null) {
26         resumen.close();
27     }
28 }

```

18.5. Resultado esperado

1	Ana	2	205.50
2	Luis	1	NULL
3	Marta	1	210.00

La suma de un grupo formado únicamente por valores NULL puede producir NULL.

18.6. Errores habituales

- Seleccionar columnas no agrupadas ni agregadas.
- Agrupar por una columna distinta de la mostrada.
- Confundir número de filas con número de valores no nulos.
- Perder clientes sin pedidos al usar INNER JOIN.
- Obtener grupos duplicados por agrupar únicamente por nombre.

18.7. Recomendación práctica

Agrupar por identificador y descripción:

```
1 GROUP BY c.id_cliente, c.nombre
```

Dos clientes distintos pueden tener el mismo nombre.

19. HAVING

19.1. ¿Qué hace?

Filtra grupos después de aplicar GROUP BY.

WHERE filtra filas antes de agrupar. HAVING filtra grupos después de agrupar.

19.2. Representación gráfica

```
1  Filas originales
2      |
3      ▼
4  WHERE
5  Filtra filas
6      |
7      ▼
8  GROUP BY
9  Construye grupos
10     |
11     ▼
12  HAVING
13  Filtra grupos
```

19.3. SQL aislado

```
1 SELECT c.id_cliente,
2        c.nombre,
3        COUNT(p.id_pedido) AS numero_pedidos,
4        SUM(p.importe) AS importe_total,
5        AVG(p.importe) AS importe_medio
6 FROM cliente c
7 INNER JOIN pedido p
```

```
8      ON p.id_cliente = c.id_cliente
9  GROUP BY c.id_cliente, c.nombre
10  HAVING COUNT(p.id_pedido) >= 2;
```

19.4. Implementación SQLJ

```
1  long minimoPedidos = 2;
2  ResumenClienteIterator resumen = null;
3
4  try {
5      #sql [contexto] resumen = {
6          SELECT c.id_cliente AS idCliente,
7                  c.nombre AS nombreCliente,
8                  COUNT(p.id_pedido) AS numeroPedidos,
9                  SUM(p.importe) AS importeTotal,
10                 AVG(p.importe) AS importeMedio
11          FROM cliente c
12          INNER JOIN pedido p
13              ON p.id_cliente = c.id_cliente
14          GROUP BY c.id_cliente, c.nombre
15          HAVING COUNT(p.id_pedido) >= :minimoPedidos
16      };
17
18      while (resumen.next()) {
19          System.out.println(
20              resumen.nombreCliente() + " | " +
21              resumen.numeroPedidos()
22          );
23      }
24  } finally {
25      if (resumen != null) {
26          resumen.close();
27      }
28  }
```

19.5. Resultado esperado

```
1  Ana | 2
```

19.6. Errores habituales

- Usar WHERE COUNT(...).
 - Aplicar en HAVING un filtro que debería estar en WHERE.
 - Agrupar demasiadas filas antes de filtrar.
 - Confundir el filtro de pedidos con el filtro de clientes.
-

19.7. Recomendación práctica

Usar:

- WHERE para condiciones de filas individuales.
- HAVING para condiciones sobre agregaciones.

Ejemplo:

```
1 WHERE p.estado <> 'CANCELADO'  
2 GROUP BY c.id_cliente, c.nombre  
3 HAVING SUM(p.importe) > 100
```

20. ORDER BY

20.1. ¿Qué hace?

Ordena el resultado de una consulta.

20.2. Representación gráfica

```
1 Sin ordenar  
2 _____  
3 210.00  
4 85.50  
5 120.00  
6      |  
7      ▼  
8 ORDER BY importe DESC  
9      |  
10     ▼  
11 210.00  
12 120.00
```

20.3. SQL aislado

```
1  SELECT id_pedido,  
2         fecha,  
3         importe,  
4         estado  
5  FROM pedido  
6  ORDER BY importe DESC, id_pedido ASC;
```

20.4. Implementación SQLJ

```
1  PedidoIterator pedidos = null;  
2  
3  try {  
4      #sql [contexto] pedidos = {  
5          SELECT id_pedido AS idPedido,  
6                 fecha AS fecha,  
7                 importe AS importe,  
8                 estado AS estado  
9          FROM pedido  
10         ORDER BY importe DESC, id_pedido ASC  
11     };  
12  
13     while (pedidos.next()) {  
14         System.out.println(  
15             pedidos.idPedido() + " | " +  
16             pedidos.importe()  
17         );  
18     }  
19 } finally {  
20     if (pedidos != null) {  
21         pedidos.close();  
22     }  
23 }
```

20.5. Resultado esperado

```
1  103 | 210.00
```

2	101		120.00
3	102		85.50
4	104		NULL

La posición de los valores NULL puede depender de la base de datos y de las opciones de ordenación disponibles.

20.6. Errores habituales

- Suponer un orden sin ORDER BY.
 - Intentar parametrizar ASC o DESC como valor.
 - Ordenar por una expresión costosa.
 - No añadir un criterio secundario estable.
 - Suponer el mismo tratamiento de NULL en todas las bases de datos.
-

20.7. Recomendación práctica

Para ordenación dinámica con SQLJ estático, elegir entre sentencias distintas:

```
1  if (ordenAscendente) {
2      #sql [contexto] pedidos = {
3          SELECT id_pedido AS idPedido,
4              fecha AS fecha,
5              importe AS importe,
6              estado AS estado
7          FROM pedido
8          ORDER BY importe ASC
9      };
10 } else {
11     #sql [contexto] pedidos = {
12         SELECT id_pedido AS idPedido,
13             fecha AS fecha,
14             importe AS importe,
15             estado AS estado
16         FROM pedido
17         ORDER BY importe DESC
18     };
19 }
```

No usar una variable host para sustituir ASC o DESC.

21. Funciones de agregación

21.1. COUNT

Explicación

Cuenta filas o valores no nulos.

- | | | |
|---|-------------------|-------------------------------------|
| 1 | COUNT(*) | → cuenta filas |
| 2 | COUNT(importe) | → cuenta importes no nulos |
| 3 | COUNT(DISTINCT x) | → cuenta valores distintos no nulos |

SQL aislado

1	SELECT COUNT(*)
2	FROM pedido;

SQLJ

1	long numeroPedidos = 0;
2	
3	#sql [contexto] {
4	SELECT COUNT(*)
5	INTO :numeroPedidos
6	FROM pedido
7	};

Resultado esperado

1	numeroPedidos = 4
---	-------------------

Error habitual

Confundir:

1	COUNT(*)
---	----------

con:

1	COUNT(importe)
---	----------------

Si existe un pedido con importe = NULL:

1	COUNT(*)	= 4
2	COUNT(importe)	= 3

Recomendación

Usar COUNT(*) para contar filas y COUNT(columna) únicamente cuando se quiera excluir NULL.

21.2. SUM

Explicación

Suma valores numéricos.

```
1  120.00 + 85.50 + 210.00 = 415.50
```

SQL aislado

```
1  SELECT SUM(importe)
2  FROM pedido;
```

SQLJ

```
1  BigDecimal importeTotal = null;
2
3  #sql [contexto] {
4      SELECT SUM(importe)
5      INTO :importeTotal
6      FROM pedido
7  };
```

Resultado esperado

```
1  importeTotal = 415.50
```

Error habitual

Suponer que devuelve cero cuando no hay valores. En muchos casos devuelve NULL.

Recomendación

Utilizar COALESCE cuando se necesite cero:

```
1  BigDecimal importeTotal = null;
2
3  #sql [contexto] {
4      SELECT COALESCE(SUM(importe), 0)
5      INTO :importeTotal
6      FROM pedido
7  };
```

21.3. AVG

Explicación

Calcula la media de los valores no nulos.

```
1 (120.00 + 85.50 + 210.00) / 3 = 138.50
```

SQL aislado

```
1 SELECT AVG(importe)
2 FROM pedido;
```

SQLJ

```
1 BigDecimal importeMedio = null;
2
3 #sql [contexto] {
4     SELECT AVG(importe)
5     INTO :importeMedio
6     FROM pedido
7 };
```

Resultado esperado

```
1 importeMedio = 138.50
```

Error habitual

Dividir mentalmente entre todas las filas, aunque alguna tenga importe = NULL.

Recomendación

Documentar si los valores ausentes deben:

- Excluirse de la media.
 - Considerarse como cero.
 - Provocar un error de calidad de datos.
-

21.4. MIN

Explicación

Obtiene el valor mínimo no nulo.

SQL aislado

```
1 SELECT MIN(importe)
2 FROM pedido;
```

SQLJ

```
1 BigDecimal importeMinimo = null;
2
3 #sql [contexto] {
4     SELECT MIN(importe)
5     INTO :importeMinimo
6     FROM pedido
7 };
```

Resultado esperado

```
1 importeMinimo = 85.50
```

Error habitual

No considerar que el resultado puede ser NULL si no existen valores.

Recomendación

Utilizar un tipo Java anulable.

21.5. MAX

Explicación

Obtiene el valor máximo no nulo.

SQL aislado

```
1 SELECT MAX(importe)
2 FROM pedido;
```

SQLJ

```
1 BigDecimal importeMaximo = null;
2
3 #sql [contexto] {
4     SELECT MAX(importe)
5     INTO :importeMaximo
6     FROM pedido
7 };
```

Resultado esperado

```
1 importeMaximo = 210.00
```

Error habitual

Confundir el importe máximo con el pedido correspondiente.

Esta consulta no obtiene directamente el pedido:

```
1 SELECT MAX(importe)
2 FROM pedido;
```

Para obtener toda la fila:

```
1 SELECT id_pedido, fecha, importe, estado
2 FROM pedido
3 WHERE importe = (
4     SELECT MAX(importe)
5     FROM pedido
6 );
```

Recomendación

Usar un iterador si puede haber varios pedidos empatados con el importe máximo.

22. Tratamiento de valores NULL

22.1. ¿Qué significa NULL?

NULL representa ausencia o desconocimiento de un valor. No es:

- Cero.
 - Una cadena vacía.
 - El texto "NULL".
 - false.
-

22.2. Comparación incorrecta

```
1 WHERE importe = NULL
```

No es la forma correcta.

Debe utilizarse:

```
1 WHERE importe IS NULL
```

o:

```
1 WHERE importe IS NOT NULL
```

22.3. Representación gráfica

```
1 importe = 0
2   |
3   └─ Valor conocido: cero
4
5 importe = NULL
6   |
7   └─ Valor ausente o desconocido
```

22.4. SQL aislado

```
1 SELECT id_pedido,
2        fecha,
3        importe,
4        estado
5 FROM pedido
6 WHERE importe IS NULL;
```

22.5. Implementación SQLJ

```
1 PedidoIterator pedidos = null;
2
3 try {
4     #sql [contexto] pedidos = {
5         SELECT id_pedido AS idPedido,
6                fecha AS fecha,
7                importe AS importe,
8                estado AS estado
9         FROM pedido
10        WHERE importe IS NULL
11     };
12
13     while (pedidos.next()) {
14         System.out.println(pedidos.idPedido());
15     }
16 }
```

```
16 } finally {
17     if (pedidos != null) {
18         pedidos.close();
19     }
20 }
```

22.6. COALESCE

COALESCE devuelve el primer valor no nulo.

```
1 SELECT COALESCE(importe, 0)
2 FROM pedido;
```

SQLJ:

```
1 BigDecimal importe = null;
2 int idPedido = 104;
3
4 #sql [contexto] {
5     SELECT COALESCE(importe, 0)
6     INTO :importe
7     FROM pedido
8     WHERE id_pedido = :idPedido
9 };
```

Resultado:

```
1 importe = 0
```

22.7. Errores habituales

- Comparar mediante = NULL.
- Mapear una columna nullable a un primitivo.
- Suponer que SUM siempre devuelve cero.
- Usar NOT IN con una subconsulta que puede devolver NULL.
- Convertir silenciosamente un valor desconocido en cero.
- Concatenar campos sin considerar el comportamiento de NULL.

22.8. Recomendación práctica

Antes de sustituir NULL por cero o por una cadena vacía, comprobar que ambos valores tienen el mismo significado funcional.

```
1 importe NULL → importe todavía no calculado
2 importe 0     → pedido gratuito
```

No son necesariamente equivalentes.

23. COMMIT

23.1. ¿Qué hace?

Confirma los cambios realizados en la transacción.

SQLJ permite ejecutar operaciones de control transaccional dentro de cláusulas ejecutables. Oracle e IBM documentan COMMIT y ROLLBACK como operaciones ejecutables SQLJ, aunque la validación previa de estas operaciones puede ser distinta de la aplicada a las sentencias DML. [\[docs.oracle.com\]](https://docs.oracle.com), [\[ibm.com\]](https://ibm.com)

23.2. Representación gráfica

```
1  INSERT cliente
2      |
3      ▼
4  Cambio pendiente
5      |
6      ▼
7      COMMIT
8      |
9      ▼
10 Cambio confirmado
```

23.3. SQL aislado

```
1 COMMIT;
```

23.4. Implementación SQLJ

```
1 #sql [contexto] {
2     COMMIT
3 };
```

También puede confirmarse mediante la conexión JDBC subyacente:

```
1 connection.commit();
```

No se deben mezclar ambas formas sin una política clara de gestión transaccional.

23.5. Resultado esperado

Los cambios pasan a formar parte de la base de datos de forma permanente y dejan de poder deshacerse mediante un ROLLBACK ordinario de esa transacción.

23.6. Errores habituales

- Confirmar después de cada sentencia.
 - Confirmar antes de terminar la operación de negocio.
 - Suponer que COMMIT nunca falla.
 - No registrar un error durante la confirmación.
 - Cerrar la conexión confiando en una confirmación implícita.
 - Mezclar control transaccional SQLJ y JDBC de forma inconsistente.
-

23.7. Recomendación práctica

Confirmar al completar una unidad de negocio coherente:

```
1 Crear pedido
2 + crear todas sus líneas
3 + calcular el total
4 + actualizar el pedido
5 = una transacción
```

24. ROLLBACK

24.1. ¿Qué hace?

Deshace los cambios pendientes de la transacción actual.

24.2. Representación gráfica

```
1 INSERT pedido
2      |
```

```

3      ▼
4  INSERT línea 1
5      |
6      ▼
7  INSERT línea 2
8      |
9      ▼
10 Error
11     |
12     ▼
13 ROLLBACK
14     |
15     ▼
16 No se conserva ni el pedido
17 ni ninguna de sus líneas

```

24.3. SQL aislado

```
1  ROLLBACK;
```

24.4. Implementación SQLJ

```

1  #sql [contexto] {
2      ROLLBACK
3  };

```

En gestión de excepciones suele ser más práctico utilizar la conexión:

```

1  try {
2      // Operaciones SQLJ.
3
4      #sql [contexto] {
5          COMMIT
6      };
7
8  } catch (SQLException exception) {
9      try {
10         #sql [contexto] {
11             ROLLBACK
12         };
13     } catch (SQLException rollbackException) {
14         exception.addSuppressed(rollbackException);

```



```
15     }  
16  
17     throw exception;  
18 }
```

24.5. Resultado esperado

Se revierten las modificaciones pendientes desde el comienzo de la transacción o desde el último COMMIT.

24.6. Errores habituales

- Ejecutar ROLLBACK con autocommit activado.
 - Intentar deshacer cambios ya confirmados.
 - Ignorar que el propio ROLLBACK puede fallar.
 - Perder la excepción original.
 - Continuar reutilizando una conexión cuyo estado es incierto.
 - No cerrar iteradores afectados por la transacción.
-

24.7. Recomendación práctica

Si falla el ROLLBACK:

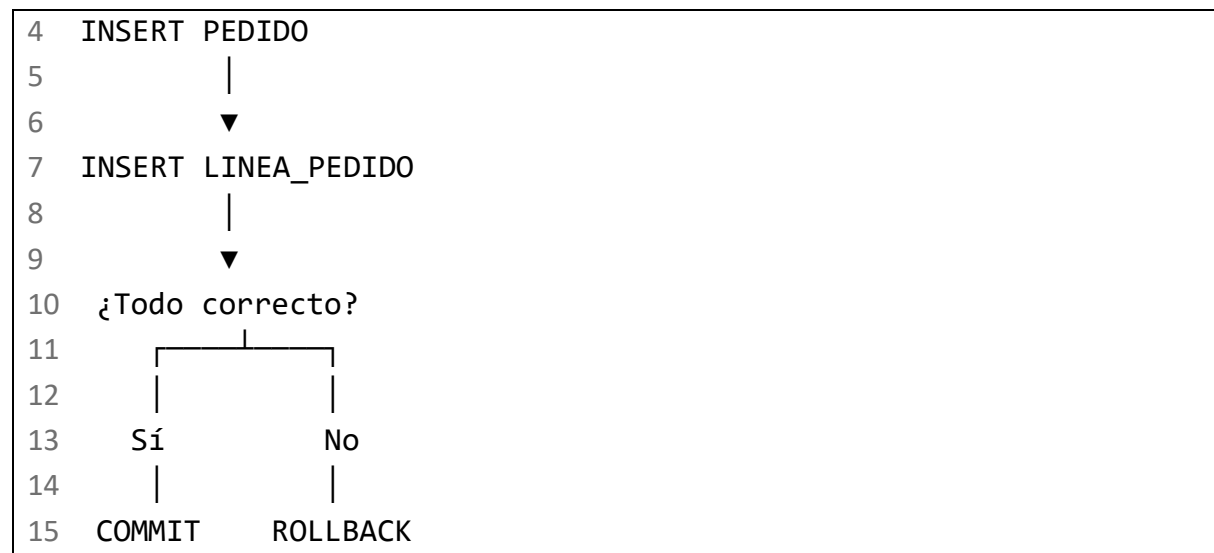
1. Conservar la excepción original.
 2. Añadir la excepción de rollback como suprimida.
 3. Considerar la conexión no reutilizable.
 4. Cerrar o invalidar la conexión.
 5. Registrar el incidente sin incluir credenciales ni datos sensibles.
-

25. Uso de transacciones

25.1. ¿Qué es una transacción?

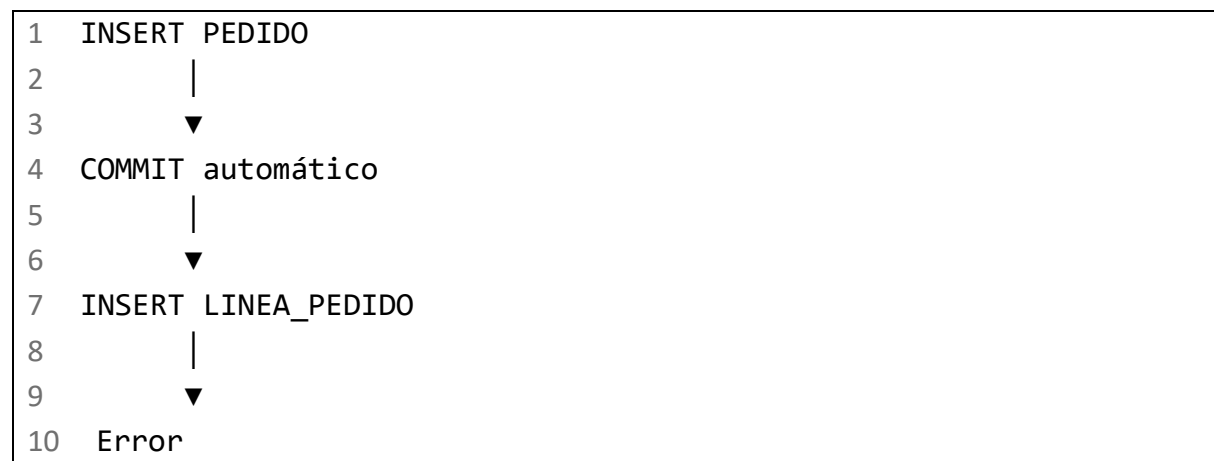
Una transacción agrupa varias operaciones que deben ejecutarse como una única unidad lógica.

```
1  Inicio de transacción  
2      |  
3      ▼
```



25.2. Autocommit

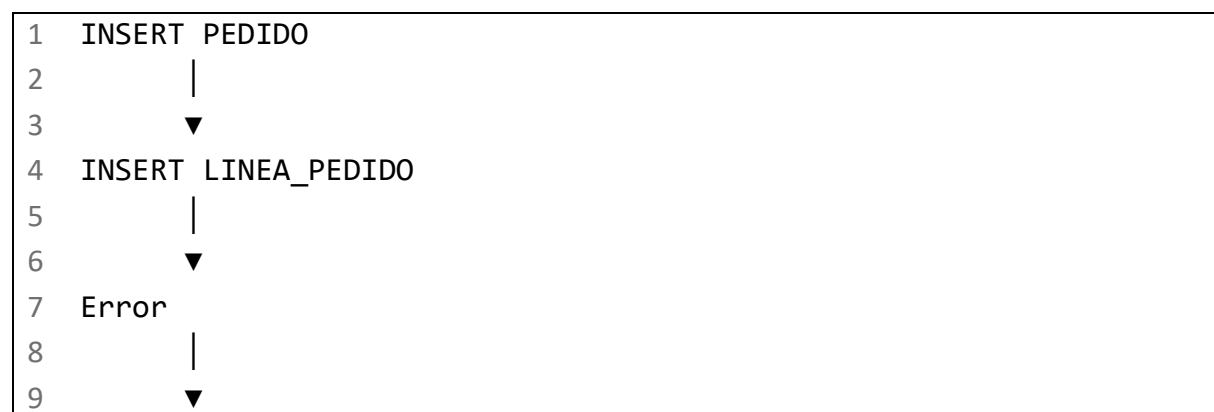
Con autocommit activado:



Resultado problemático:

- 1 El pedido existe,
- 2 pero sus líneas no.

Con autocommit desactivado:



25.3. Ejemplo SQLJ

```
1  import java.math.BigDecimal;
2  import java.sql.Connection;
3  import java.sql.Date;
4  import java.sql.SQLException;
5
6  public void crearPedido(
7      Connection connection,
8      ContextoAplicacion contexto,
9      int idPedido,
10     int idCliente,
11     Date fecha,
12     BigDecimal importe,
13     String estado,
14     int idProducto,
15     int cantidad) throws SQLException {
16
17     boolean autoCommitOriginal = connection.setAutoCommit();
18
19     try {
20         connection.setAutoCommit(false);
21
22         #sql [contexto] {
23             INSERT INTO pedido (
24                 id_pedido,
25                 id_cliente,
26                 fecha,
27                 importe,
28                 estado
29             )
30             VALUES (
31                 :idPedido,
32                 :idCliente,
33                 :fecha,
34                 :importe,
35                 :estado
36             )
```

```

37         };
38
39         #sql [contexto] {
40             INSERT INTO linea_pedido (
41                 id_pedido,
42                 id_producto,
43                 cantidad
44             )
45             VALUES (
46                 :idPedido,
47                 :idProducto,
48                 :cantidad
49             )
50         };
51
52         #sql [contexto] {
53             COMMIT
54         };
55
56     } catch (SQLException exception) {
57         try {
58             #sql [contexto] {
59                 ROLLBACK
60             };
61         } catch (SQLException rollbackException) {
62             exception.addSuppressed(rollbackException);
63         }
64
65         throw exception;
66
67     } finally {
68         try {
69             connection.setAutoCommit(autoCommitOriginal);
70         } catch (SQLException restoreException) {
71             // En una aplicación real debe invalidarse la
conexión
72             // si no puede restaurarse su estado
correctamente.
73             throw restoreException;
74         }
75     }

```

25.4. Aspectos a vigilar

La conexión y el contexto deben representar la misma unidad de trabajo

No debe desactivarse el autocommit en una conexión y ejecutar SQLJ en otra.

```
1 connectionA.setAutoCommit(false)
2
3 contexto → connectionB
4
5 Resultado: control transaccional incorrecto
```

El COMMIT puede fallar

Una transacción no debe considerarse confirmada hasta que el COMMIT termine correctamente.

Restaurar el estado de la conexión

Si una conexión procede de un pool:

- Restaurar autoCommit.
- Cerrar iteradores.
- Deshacer cambios pendientes.
- No devolver al pool una conexión en estado incierto.

26. Prevención de modificaciones masivas accidentales

26.1. UPDATE peligroso

```
1 UPDATE pedido
2 SET estado = 'CANCELADO';
```

Resultado:

```
1 Todos los pedidos quedan cancelados
```

26.2. DELETE peligroso

```
1 DELETE FROM pedido;
```

Resultado:

1 Se eliminan todos los pedidos

26.3. Patrón más seguro

```
1  int idPedido = 101;
2  String estadoEsperado = "PENDIENTE";
3  String nuevoEstado = "CANCELADO";
4
5  #sql [contexto] {
6      UPDATE pedido
7      SET estado = :nuevoEstado
8      WHERE id_pedido = :idPedido
9      AND estado = :estadoEsperado
10 };
```

26.4. Comprobación del número de filas afectadas

SQLJ dispone de `ExecutionContext` para consultar información sobre la ejecución, incluido el recuento de actualización, aunque la utilización concreta debe verificarse con la implementación. IBM documenta el uso de `ExecutionContext` para controlar o supervisar sentencias SQLJ. [\[ibm.com\]](#), [\[ibm.com\]](#)

Patrón conceptual:

```
1  sqlj.runtime.ExecutionContext executionContext =
2      new sqlj.runtime.ExecutionContext();
3
4  #sql [contexto, executionContext] {
5      UPDATE pedido
6      SET estado = :nuevoEstado
7      WHERE id_pedido = :idPedido
8      AND estado = :estadoEsperado
9  };
10
11  int filasAfectadas = executionContext.getUpdateCount();
12
13  if (filasAfectadas != 1) {
14      throw new SQLException(
15          "Se esperaba actualizar un pedido, pero se
16      actualizaron: " +
17          filasAfectadas
```

```

17     );
18 }

```

La firma concreta de los métodos y su comportamiento deben comprobarse en la implementación SQLJ seleccionada.

27. Resumen comparativo de operaciones

Operación	Finalidad	Cardinalidad habitual	Riesgo principal
SELECT INTO	Recuperar una fila	Exactamente 1	Cero o varias filas
SELECT con iterador	Recuperar varias filas	0..N	No cerrar el iterador
INSERT	Crear una fila	1	Restricciones y duplicados
UPDATE	Modificar filas	0..N	Actualización masiva
DELETE	Eliminar filas	0..N	Borrado masivo o claves ajenas
INNER JOIN	Coincidencias	0..N	Relaciones mal definidas
LEFT JOIN	Conservar tabla izquierda	0..N	No tratar NULL
RIGHT JOIN	Conservar tabla derecha	0..N	Portabilidad y legibilidad
UNION	Combinar sin duplicados	0..N	Coste de eliminar duplicados
UNION ALL	Combinar con duplicados	0..N	Duplicados no deseados
EXISTS	Comprobar existencia	0..N	Mala correlación
NOT EXISTS	Comprobar ausencia	0..N	Mala correlación
GROUP BY	Agrupar filas	0..N grupos	Agrupación incorrecta
HAVING	Filtrar grupos	0..N grupos	Confundirlo con WHERE
COMMIT	Confirmar cambios	Transacción	Confirmación prematura
ROLLBACK	Deshacer cambios	Transacción	No poder deshacer un commit

28. Errores comunes de esta entrega

28.1. Usar SELECT INTO para varias filas

```

1 #sql [contexto] {
2     SELECT nombre

```

```
3      INTO :nombre
4      FROM cliente
5      WHERE ciudad = :ciudad
6  };
```

Si existen varios clientes en esa ciudad, debe emplearse un iterador.

28.2. Confundir variables host con fragmentos SQL

Incorrecto:

```
1  String orden = "DESC";
2
3  #sql [contexto] pedidos = {
4      SELECT ...
5      FROM pedido
6      ORDER BY importe :orden
7  };
```

Una variable host representa un valor, no la palabra SQL DESC.

28.3. No tratar columnas nullable

Problemático:

```
1  int idPedido;
```

si procede de la parte opcional de un LEFT JOIN.

Correcto:

```
1  Integer idPedido;
```

28.4. No cerrar iteradores

```
1  PedidoIterator pedidos = null;
2
3  try {
4      // Ejecutar y recorrer.
5  } finally {
6      if (pedidos != null) {
7          pedidos.close();
8      }
9  }
```

28.5. Usar WHERE en vez de ON con un LEFT JOIN

```
1 LEFT JOIN pedido p
2     ON p.id_cliente = c.id_cliente
3 WHERE p.estado = 'PENDIENTE'
```

Esto elimina los clientes sin pedidos.

Si se quieren conservar:

```
1 LEFT JOIN pedido p
2     ON p.id_cliente = c.id_cliente
3     AND p.estado = 'PENDIENTE'
```

28.6. Ejecutar operaciones relacionadas con autocommit

Crear un pedido y sus líneas exige una transacción conjunta.

```
1 Pedido sin líneas
2 o
3 Líneas sin pedido
```

son estados que la operación debe impedir.

29. Mapa visual de operaciones

```
1 OPERACIONES SQL EN SQLJ
2 |
3 |— Consulta de datos
4 |   |— SELECT INTO
5 |   |   |— Exactamente una fila
6 |   |
7 |   |— Iteradores
8 |   |   |— Cero, una o varias filas
9 |
10 |— Modificación
11 |   |— INSERT
12 |   |— UPDATE
13 |   |— DELETE
14 |
15 |— Combinación de tablas
```

16			INNER JOIN
17			LEFT JOIN
18			RIGHT JOIN
19			JOIN de múltiples tablas
20			
21			Combinación de resultados
22			UNION
23			└─ Elimina duplicados
24			UNION ALL
25			└─ Conserva duplicados
26			
27			Filtros avanzados
28			Subconsultas
29			EXISTS
30			NOT EXISTS
31			IN
32			└─ NOT IN
33			
34			Agrupación
35			GROUP BY
36			HAVING
37			COUNT
38			SUM
39			AVG
40			MIN
41			└─ MAX
42			
43			Ordenación
44			└─ ORDER BY
45			
46			Valores ausentes
47			IS NULL
48			IS NOT NULL
49			└─ COALESCE
50			
51			Transacciones
52			Autocommit
53			COMMIT
54			└─ ROLLBACK

30. Lista de comprobación

Después de esta segunda entrega, el alumno debería dominar:

- Recuperación de una fila mediante SELECT INTO.
- Recuperación de varias filas mediante iteradores.
- Inserción de datos con variables host.
- Actualización segura de filas.
- Eliminación respetando claves ajenas.
- Diferencia entre INNER JOIN y LEFT JOIN.
- Funcionamiento y posibles limitaciones de RIGHT JOIN.
- Construcción de un JOIN con cuatro tablas.
- Diferencia entre UNION y UNION ALL.
- Requisitos de compatibilidad entre consultas combinadas.
- Utilización de subconsultas escalares.
- Diferencia entre EXISTS, NOT EXISTS, IN y NOT IN.
- Riesgos de NOT IN con valores NULL.
- Agrupación mediante GROUP BY.
- Diferencia entre WHERE y HAVING.
- Uso de COUNT, SUM, AVG, MIN y MAX.
- Tratamiento de valores NULL.
- Diferencia entre ON y WHERE en un LEFT JOIN.
- Ordenación mediante ORDER BY.
- Control de transacciones con COMMIT y ROLLBACK.
- Riesgo de utilizar autocommit en operaciones compuestas.
- Prevención de actualizaciones y eliminaciones masivas.
- Necesidad de cerrar los iteradores.
- Identificación de construcciones dependientes del fabricante.

La tercera entrega desarrollará los ejemplos Java completos, la gestión detallada de transacciones, los códigos de error, las excepciones, el cierre de recursos y los diez casos prácticos definidos en la guía.